

AWS Key Management Service (KMS) & AWS Secrets Manager

What is AWS KMS?

AWS Key Management Service (KMS) is a **managed service** used to **create and manage encryption keys**.

It helps protect data by **encrypting and decrypting** it securely.

Use Cases of AWS KMS

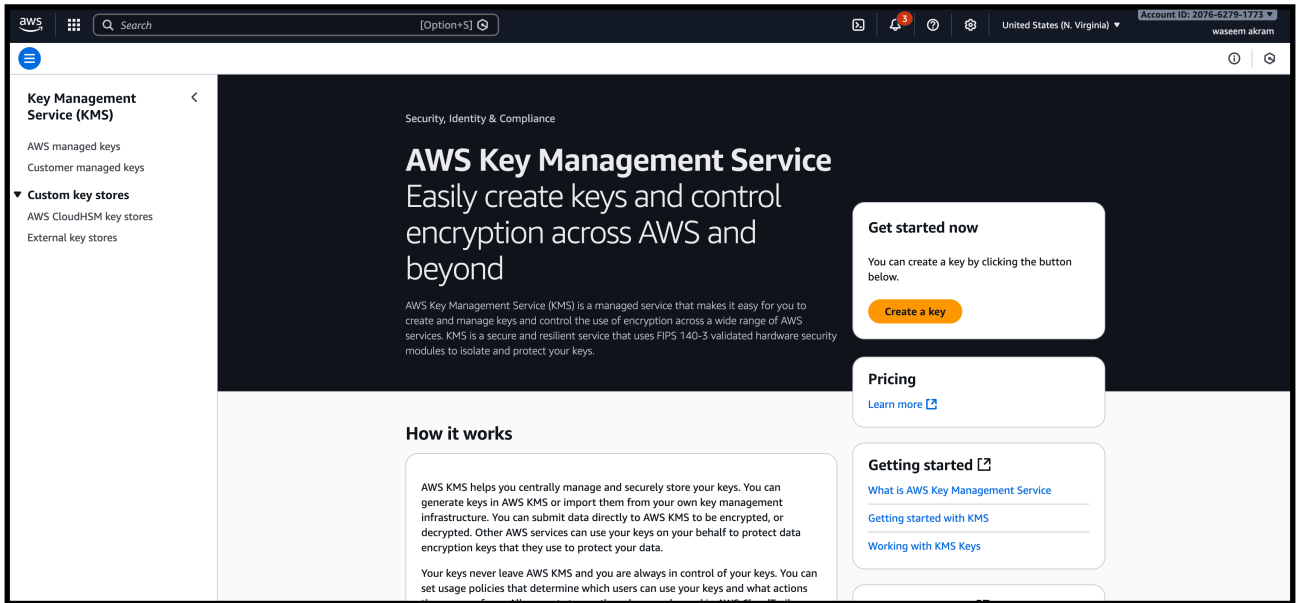
- Encrypt data stored in **S3, EBS, RDS**
- Encrypt application data
- Secure sensitive information
- Meet security and compliance requirements
- Centralized key management

Task 1: encrypt a s3 bucket using the kms key

Execution Steps – Create a KMS Key

Step 1: Open AWS KMS

- Go to **AWS Console**
- Search for **Key Management Service (KMS)**
- Click **Customer managed keys**
- Click **Create key**



Step 2: Configure Key

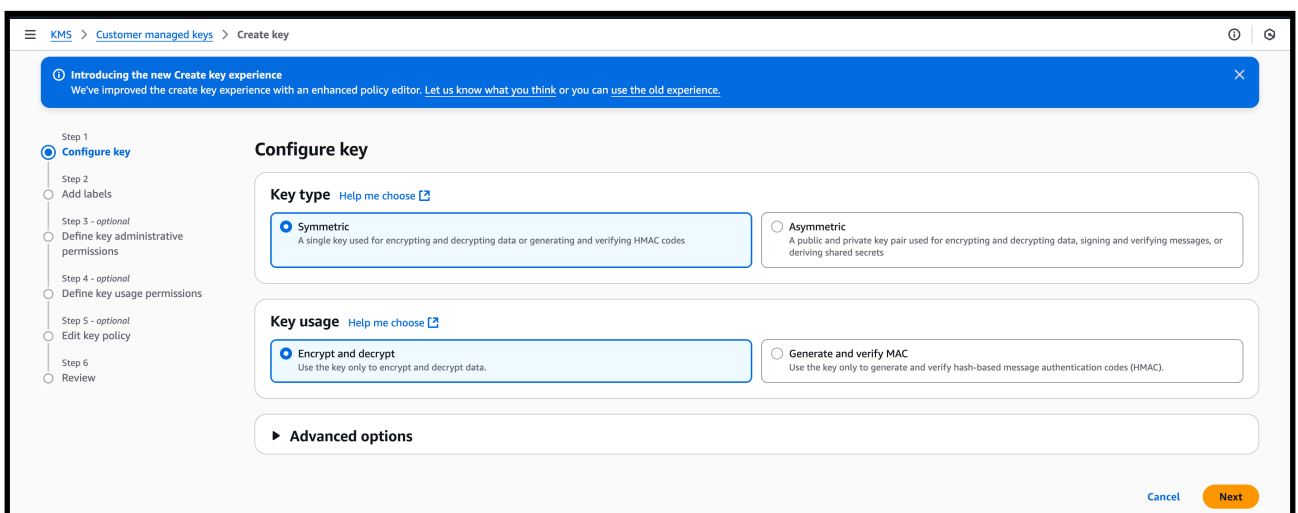
Key Type

- Select: **Symmetric**

Key Usage

- Select: **Encrypt and decrypt**

➡ Click **Next**



Step 3: Add Labels

Alias

- Enter alias:

my-kms-key

The screenshot shows the AWS KMS 'Create key' wizard. The breadcrumb navigation at the top reads 'KMS > Customer managed keys > Create key'. A blue banner at the top of the main content area says 'Introducing the new Create key experience' and provides a link to learn more. On the left, a vertical list of steps shows 'Step 2: Add labels' as the current step, with other steps like 'Configure key', 'Define key administrative permissions', and 'Review' listed below it. The main content area is titled 'Add labels' and contains three sections: 'Alias', 'Description - optional', and 'Tags - optional'. The 'Alias' section has a text input field containing 'my-kms-key'. The 'Description' section has a text area with the placeholder 'Description of the key'. The 'Tags' section has an 'Add tag' button and a note that you can add up to 50 tags. At the bottom right, there are four buttons: 'Cancel', 'Skip to Review', 'Previous', and 'Next'.

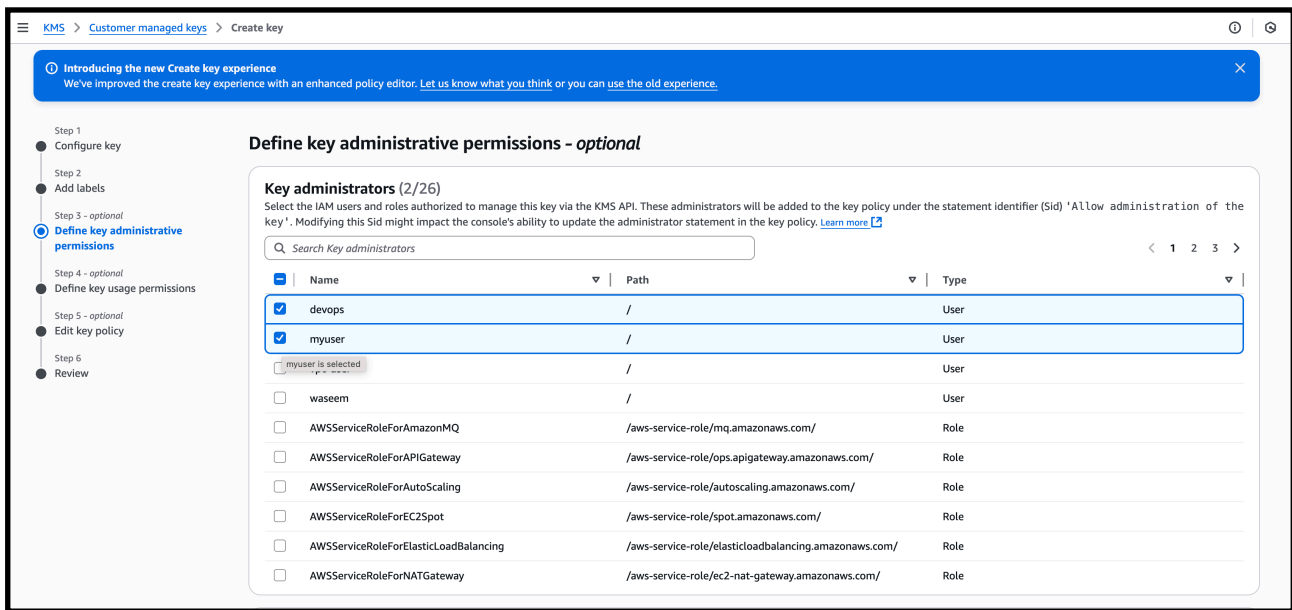
Step 4: Define Key Administrative Permissions

Select users who can **manage the key** (enable/disable, rotate, delete).

Selected:

- devops
- myuser

➡ Click Next

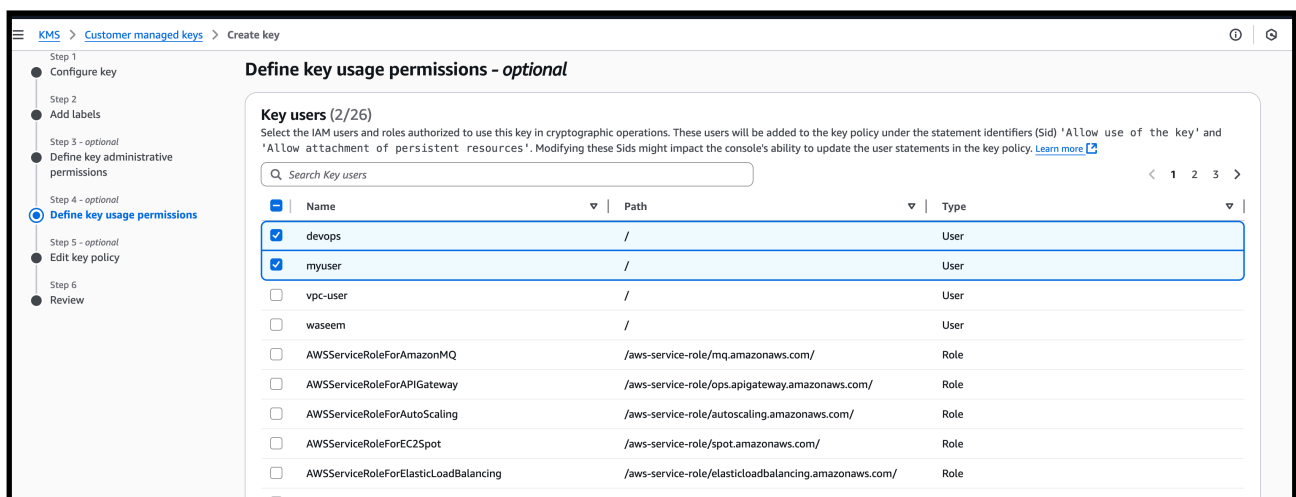


Step 5: Define Key Usage Permissions

Select users who can **use the key** for encryption and decryption.

Selected:

- devops
- myuser



➡ Click Next

—> keep the policy as it is

The screenshot shows the AWS KMS 'Create key' console. At the top, there's a navigation bar with 'KMS > Customer managed keys > Create key'. Below this is a blue banner with an information icon and text: 'Introducing the new Create key experience. We've improved the create key experience with an enhanced policy editor. [Let us know what you think](#) or you can [use the old experience](#).' On the left, a vertical progress bar shows six steps: 'Step 1: Configure key', 'Step 2: Add labels', 'Step 3 - optional: Define key administrative permissions', 'Step 4 - optional: Define key usage permissions', 'Step 5 - optional: Edit key policy' (which is selected with a blue circle), and 'Step 6: Review'. The main area is titled 'Edit key policy - optional' and contains a 'Key policy' section. It instructs the user to 'Review the key policy statements for this key. To manually update this policy, select **Edit**. Modifying the statement identifier updates to that statement.' Below this is a code editor showing two JSON policy statements. The first statement, 'Enable IAM User Permissions', allows the root user to perform all KMS actions on all resources. The second statement, 'Allow access for Key Administrators', allows two specific IAM users ('devops' and 'myuser') to perform all KMS actions on all resources.

```
1  {
2    "Id": "key-consolepolicy-3",
3    "Version": "2012-10-17",
4    "Statement": [
5      {
6        "Sid": "Enable IAM User Permissions",
7        "Effect": "Allow",
8        "Principal": {
9          "AWS": "arn:aws:iam::207662791773:root"
10       },
11       "Action": "kms:*",
12       "Resource": "*"
13     },
14     {
15       "Sid": "Allow access for Key Administrators",
16       "Effect": "Allow",
17       "Principal": {
18         "AWS": [
19           "arn:aws:iam::207662791773:user/devops",
20           "arn:aws:iam::207662791773:user/myuser"
21         ]
22       }
23     }
24   ]
25 }
```

—> click next

Step 7: Review and Create

Verify:

- Key type: **Symmetric**
- Key usage: **Encrypt and decrypt**
- Alias: **my-kms-key**
- Administrators: **devops, myuser**
- Users: **devops, myuser**

➡ Click **Create key**

Key configuration

Key type Symmetric	Key spec SYMMETRIC_DEFAULT	Key usage Encrypt and decrypt
Origin AWS KMS	Regionality Single-Region key	

You cannot change the key configuration after the key is created.

Alias and description

Alias my-kms-key	Description -
----------------------------	-------------------------

Tags

Key	Value
No data No tags to display	

Key administrators

The following are IAM users and roles added to the key policy under 'Allow' administration of the key. If you modified this Sid, or added administrators under a different Sid, they will not be included in this list.

Success
Your AWS KMS key was created with alias my-kms-key and key ID 100b82f9-6f95-4804-9952-71a050d57c77.

Customer managed keys (1)

Filter keys by properties or tags

Aliases	Key ID	Status	Key type	Key spec
my-kms-key	100b82f9-6f95-4804-9952-71a...	Enabled	Symmetric	SYMMETRIC_DEFAULT

- KMS key created successfully
- Alias: **my-kms-key**
- Status: **Enabled**
- Key type: **Symmetric**
- Key spec: **SYMMETRIC_DEFAULT**

This key is now available for:

- S3 encryption
- EBS encryption
- Secrets Manager
- Other AWS services

—-> now create a s3 bucket

Amazon S3

>

Buckets

>

Create bucket

You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#)

You can use s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation. Please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. [Learn more](#)

No tags associated with this bucket.

Add new tag

You can add up to 50 tags.

Default encryption

Info

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type

Info

Secure your objects with two separate layers of encryption. For details on pricing, see DSSE-KMS pricing on the Storage tab of the [Amazon S3 pricing page](#).

☐ Server-side encryption with Amazon S3 managed keys (SSE-S3)

☒ Server-side encryption with AWS Key Management Service keys (SSE-KMS)

☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

AWS KMS key

Info

☒ Choose from your AWS KMS keys

☐ Enter AWS KMS key ARN

Available AWS KMS keys

Choose AWS KMS key

arn:aws:kms:us-east-1:207662791773:key/100b82f9-6f95-4804-9952-71a050d57c77
my-kms-key

arn:aws:kms:us-east-1:207662791773:a
aws/s3

arn:aws:kms:us-east-1:207662791773:key/
100b82f9-6f95-4804-9952-71a050d57c77

Create a KMS key

[learn more](#)

—> give the aws kms keys which we created : my-kms-key is the name of key which we created

Default encryption [Info](#)

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [Info](#)

Secure your objects with two separate layers of encryption. For details on pricing, see [DSSE-KMS pricing](#) on the [Storage](#) tab of the [Amazon S3 pricing page](#). [L](#)

☐ Server-side encryption with Amazon S3 managed keys (SSE-S3)

☒ Server-side encryption with AWS Key Management Service keys (SSE-KMS)

☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

AWS KMS key [Info](#)

☒ Choose from your AWS KMS keys

☐ Enter AWS KMS key ARN

Available AWS KMS keys

arn:aws:kms:us-east-1:207662791773:key/100b82f9-6f95-4804-9952-71a050d57c77



Create a KMS key [L](#)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#) [L](#)

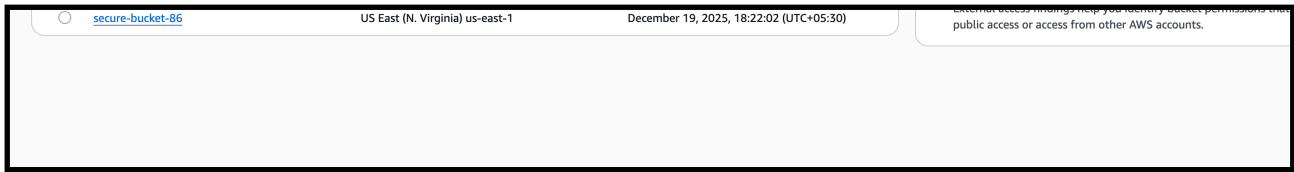
☐ Disable

☒ Enable

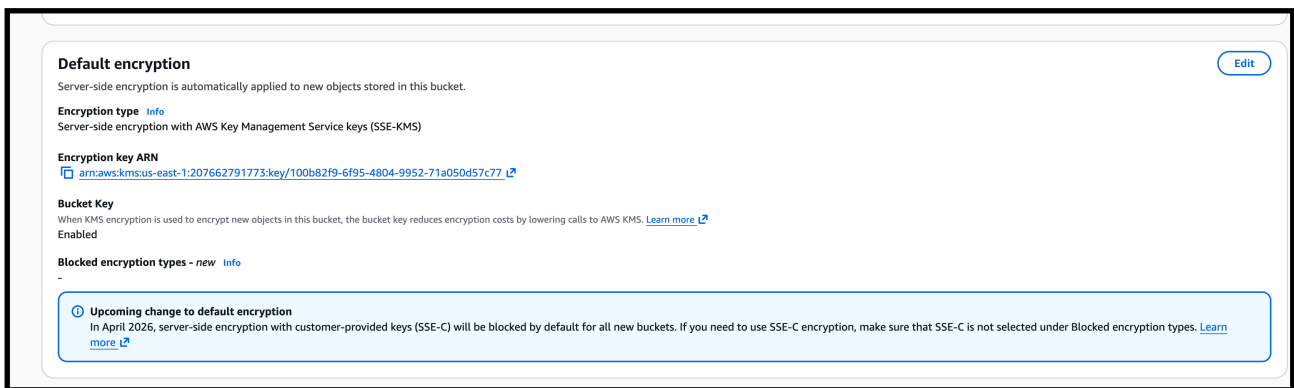
► Advanced settings

ⓘ

After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.



—> s3 bucket is successfully with awk encryption role attached



Secrets manager

What is AWS Secrets Manager?

AWS Secrets Manager is a service used to **store sensitive information securely**.

Sensitive information means:

- Database username & password
- API keys
- Tokens
- Application secrets

Why we need it?

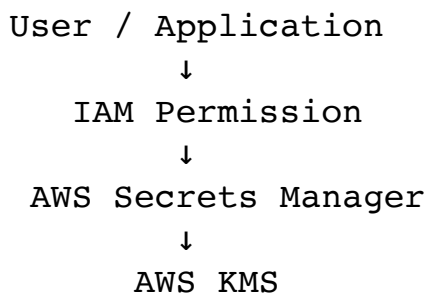
- >Secrets Manager stores them **encrypted**
- >Only allowed users/services can read them
- >Uses **AWS KMS** for encryption

Secrets Manager = Safe locker for passwords in AWS

Use Case:

An application needs a database username and password.
Instead of storing it in code, we store it securely in **Secrets Manager**.

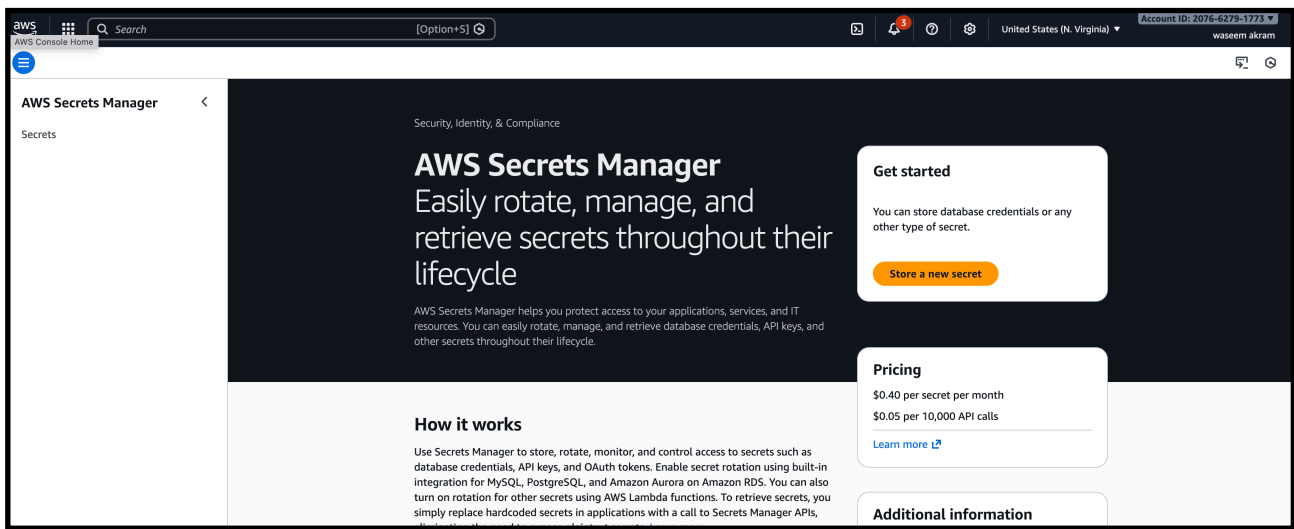
Architecture Flow (Simple)



Task: Store and Retrieve Database Credentials using AWS Secrets Manager

Step 1: Open Secrets Manager

1. Login to **AWS Management Console**
2. Search for **Secrets Manager**
3. Click **Store a new secret**



Step 2: Choose Secret Type

(As shown in your screenshot)

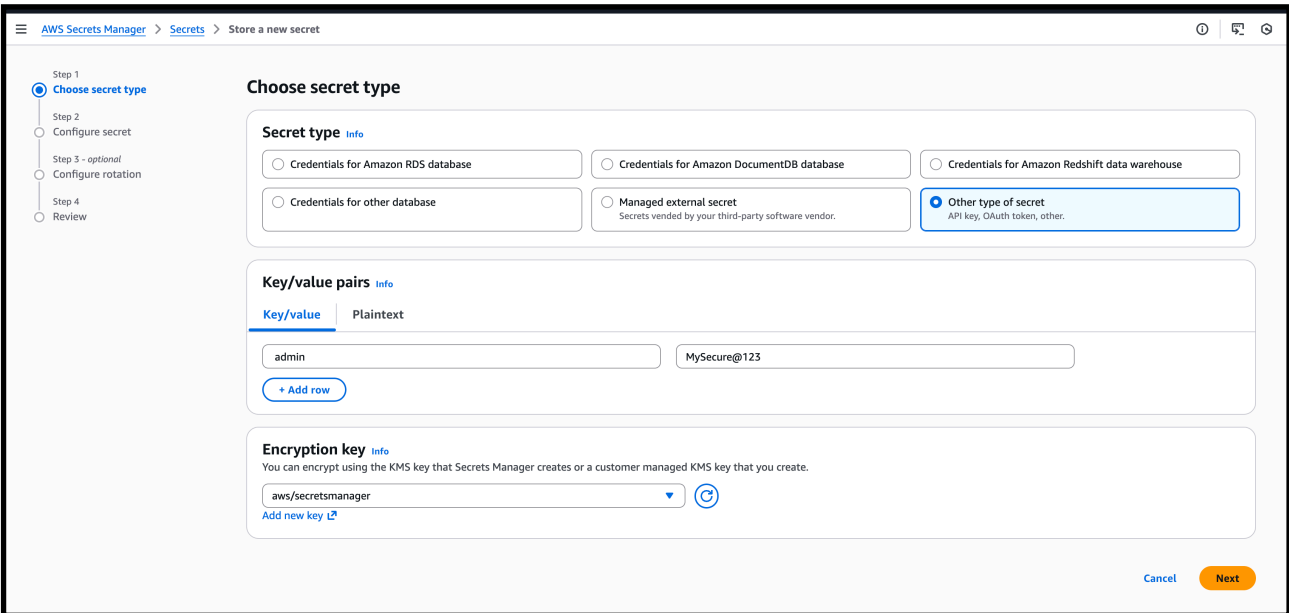
- Select **Other type of secret**
(Used for API keys, usernames, passwords, tokens, etc.)

Step 3: Add Key / Value Pairs

Under **Key/value pairs**:

Key	Value
admin	MySecure@123

- Click **Add row** if you want to add more secrets
- Data can also be entered in **Plaintext** format (optional)



These values will be stored securely and encrypted.

Select Encryption Key

Under **Encryption key**:

- Choose **aws/secretsmanager**
(Default AWS managed KMS key)

This ensures the secret is **encrypted using AWS KMS**.

—> Proceed to Next Step

The screenshot shows the AWS Secrets Manager console interface for creating a new secret. The breadcrumb navigation at the top reads 'AWS Secrets Manager > Secrets > Store a new secret'. On the left, a vertical progress bar indicates four steps: 'Step 1 Choose secret type' (completed), 'Step 2 Configure secret' (active), 'Step 3 - optional Configure rotation' (skipped), and 'Step 4 Review' (skipped). The main content area is titled 'Configure secret'. It contains two sections: 'Secret name and description' and 'Tags - optional'. The 'Secret name' section has a text input field containing 'my-db-credentials' and a description field containing 'Access to MYSQL prod database for my AppBeta'. The 'Tags' section shows 'No tags associated with the secret' and an 'Add' button.

Step 1 Choose secret type

Step 2 **Configure secret**

Step 3 - optional Configure rotation

Step 4 Review

Configure secret

Secret name and description [Info](#)

Secret name
A descriptive name that helps you find your secret later.

my-db-credentials

Secret name must contain only alphanumeric characters and the characters /_+=.@-

Description - optional

Access to MYSQL prod database for my AppBeta

Maximum 250 characters.

Tags - optional

No tags associated with the secret.

Add

Step 4: Name the Secret

- **Secret name:** my-db-credentials
- **Description:** Database credentials for application

Step 5: Review and Create

- Check all details
- Click **Store**

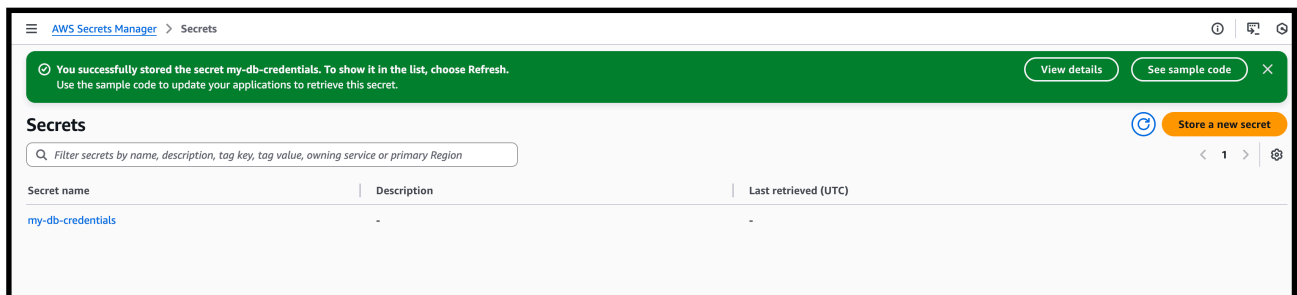
The screenshot shows the 'Store a new secret' wizard in the AWS console, specifically the 'Review' step. On the left, a progress bar indicates the steps: Step 1 (Choose secret type), Step 2 (Configure secret), Step 3 - optional (Configure rotation), and Step 4 (Review), which is currently selected. The main area is titled 'Review' and contains two sections: 'Secret type' and 'Secret configuration'. The 'Secret type' section shows 'Secret type' as 'Other type of secret' and 'Encryption key' as 'aws/secretsmanager'. The 'Secret configuration' section shows 'Secret name' as 'my-db-credentials', 'Description' as '-', 'Tags' as '-', 'Resource permissions' as '-', and 'Secret replication' as '-'. The breadcrumb at the top reads 'AWS Secrets Manager > Secrets > Store a new secret'.

The screenshot shows the 'Sample code' section of the AWS console. It features a tabbed interface with tabs for Java, JavaScript, C#, Python3, Ruby, Go, Rust, and PHP. The 'Java' tab is active, displaying a code editor with the following Java code:

```
25
26 GetSecretValueResponse getSecretValueResponse;
27
28 try {
29     getSecretValueResponse = client.getSecretValue(getSecretValueRequest);
30 } catch (Exception e) {
31     // For a list of exceptions thrown, see
32     // https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
33     throw e;
34 }
35
36 String secret = getSecretValueResponse.secretString();
37
38 // Your code goes here.
39 }
```

Below the code editor, there is a status bar showing 'Java Line 1, Column 1' and 'Errors: 0 Warnings: 0'. A link to 'Download AWS SDK for Java' is provided. At the bottom right of the page, there are three buttons: 'Cancel', 'Previous', and 'Store'.

—> successfully created a secret for aws secret manager



Step 6: Retrieve Secret (Verification)

1. Open secret **my-db-credentials**
2. Click **Retrieve secret value**

You will see:

```
{
  "username": "admin",
  "password": "MySecure@123"
}
```

